

ENTREGA SPRINT

FEDERICO URBINA

Curso especialización

IT ACADEMY



Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Operaciones realizadas:

Creo una nueva BBDD llamada Sprint04

```
1  -- Creo la BBDD
2  • CREATE DATABASE sprint04;
3  • USE sprint04;
4
```

ut

Action Output ▼

	Time	Action
1	14:31:23	CREATE DATABASE sprint04
2	14:31:46	USE sprint04

Operaciones realizadas:

- Revisé los archivos CSV para identificar los campos disponibles.
- Creé la base de datos sprint04.
- Creé las tablas users, companies, credit_cards, products y transactions.
- Definí una única tabla users para cargar los datos de american_users.csv y european_users.csv.
- Definí claves primarias en cada tabla según el identificador de los datos.

```

7 • CREATE TABLE users (
8     id VARCHAR(250) NOT NULL,
9     name VARCHAR(250),
10    surname VARCHAR(250),
11    phone VARCHAR(250),
12    email VARCHAR(250),
13    birth_date VARCHAR(250),
14    country VARCHAR(250),
15    city VARCHAR(250),
16    postal_code VARCHAR(250),
17    address VARCHAR(250),
18    PRIMARY KEY (id)
19 );
20
21 • CREATE TABLE companies (
22     company_id VARCHAR(250) NOT NULL,
23     company_name VARCHAR(250),
24     phone VARCHAR(250),
25     email VARCHAR(250),
26     country VARCHAR(250),
27     website VARCHAR(250),
28     PRIMARY KEY (company_id)
29 );
30
31 • CREATE TABLE credit_cards (
32     id VARCHAR(250) NOT NULL,
33     user_id VARCHAR(250),
34     iban VARCHAR(250),
35     pan VARCHAR(250),
36     pin VARCHAR(250),
37     cvv VARCHAR(250),
38     track1 VARCHAR(250),
39     track2 VARCHAR(250),
40     expiring_date VARCHAR(250),
41     PRIMARY KEY (id)
42 );

```

```

43
44 • CREATE TABLE products (
45     id VARCHAR(250) NOT NULL,
46     product_name VARCHAR(250),
47     price VARCHAR(250),
48     colour VARCHAR(250),
49     weight VARCHAR(250),
50     warehouse_id VARCHAR(250),
51     PRIMARY KEY (id)
52 );
53
54 • CREATE TABLE transactions (
55     id VARCHAR(250) NOT NULL,
56     card_id VARCHAR(250),
57     business_id VARCHAR(250),
58     timestamp VARCHAR(250),
59     amount VARCHAR(250),
60     declined VARCHAR(250),
61     product_ids VARCHAR(250),
62     user_id VARCHAR(250),
63     lat VARCHAR(250),
64     longitude VARCHAR(250),
65     PRIMARY KEY (id)
66 );
67

```

Output

#	Time	Action
1	14:47:22	CREATE TABLE users (id VARCHAR(250) NOT NU
2	14:47:55	CREATE TABLE companies (company_id VARCHA
3	14:48:23	CREATE TABLE products (id VARCHAR(250) NOT
4	14:48:32	CREATE TABLE transactions (id VARCHAR(250) N

Resultado clave:

- La base de datos queda estructurada con todas las tablas necesarias para la carga de datos.


```

74 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/companies 2.csv'
75 INTO TABLE companies
76 FIELDS TERMINATED BY ','
77 LINES TERMINATED BY '\r\n'
78 IGNORE 1 LINES
79 (company_id, company_name, phone, email, country, website);
80
81 • SELECT count(1) FROM companies;
82
83 •

```

00% 60:79

Action Output

	Time	Action
9	12:44:28	SELECT * FROM sprint04.companies LIMIT 0, 50000

```

83 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/american_users.csv'
84 INTO TABLE users
85 FIELDS TERMINATED BY ','
86 ENCLOSED BY '"'
87 LINES TERMINATED BY '\n'
88 IGNORE 1 LINES
89 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
90
91 • SELECT COUNT(1) FROM users; -- Verifico si están los
92
93 •

```

100% 55:91

Result Grid

COUNT(1)
1010

Result 1

Action Output

	Time	Action	Response
1	12:59:16	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/american_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES...	1010 row(s) a...
2	13:02:47	SELECT COUNT(1) FROM users LIMIT 0, 50000	1 row(s) retur...

```

93 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/european_users.csv'
94 INTO TABLE users
95 FIELDS TERMINATED BY ','
96 ENCLOSED BY '"'
97 LINES TERMINATED BY '\n'
98 IGNORE 1 LINES
99 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
100
101 • SELECT COUNT(1) FROM users; -- 5000 usuarios es la suma de los 1010 Americanos + 3990 de Europeos
102

```

100% 98:101

Result Grid

COUNT(1)
5000

Result 2

Action Output

	Time	Action	Response
3	13:06:17	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/european_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES...	3990 row(s)...
4	13:06:56	SELECT COUNT(1) FROM users LIMIT 0, 50000	1 row(s) retur...

```
103 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/credit_cards.csv'
104 INTO TABLE credit_cards
105 FIELDS TERMINATED BY ','
106 ENCLOSED BY '"'
107 LINES TERMINATED BY '\n'
108 IGNORE 1 LINES;
109
110 • SELECT COUNT(1) FROM credit_cards;
111
```

00% 35:110

Result Grid Filter Rows: Search Export:

COUNT(1)
5000

Result 3

Action Output

	Time	Action	Response
5	13:12:26	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/credit_cards.csv' INTO TABLE credit_cards FIELDS TERMINATED BY ',' ENCLOSED BY '"' LI...	5000 row(s)...
6	13:12:34	SELECT COUNT(1) FROM credit_cards LIMIT 0, 50000	1 row(s) retur...

```
112 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/products.csv'
113 INTO TABLE products
114 FIELDS TERMINATED BY ','
115 ENCLOSED BY '"'
116 LINES TERMINATED BY '\n'
117 IGNORE 1 LINES;
118
119 • SELECT COUNT(*) AS Prod FROM products;
120
```

100% 39:119

Result Grid Filter Rows: Search Export:

Prod
100

Result 4

Action Output

	Time	Action	Response
7	13:15:14	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/products.csv' INTO TABLE products FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TE...	100 row(s) af...
8	13:15:30	SELECT COUNT(*) AS Prod FROM products LIMIT 0, 50000	1 row(s) retur...

```
121 • LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/transactions.csv'
122 INTO TABLE transactions
123 FIELDS TERMINATED BY ';'
124 ENCLOSED BY '"'
125 LINES TERMINATED BY '\n'
126 IGNORE 1 LINES;
127
128 • SELECT COUNT(*) AS TOTAL_TRANSACTIONS
129 FROM transactions;
130
```

100% 19:129

Result Grid Filter Rows: Search Export:

TOTAL_TRANSACTIONS
100000

Result 5

Action Output

	Time	Action	Response
1	13:25:03	LOAD DATA LOCAL INFILE '/Users/fedeur/IT Academy/SQL/4 Sprint/BBDD/transactions.csv' INTO TABLE t...	100000 row(s)...
2	13:25:08	SELECT COUNT(*) AS TOTAL_TRANSACTIONS FROM transactions LIMIT 0, 50000	1 row(s) retur...

```
131 • ALTER TABLE credit_cards
132     ADD CONSTRAINT fk_credit_cards_users
133     FOREIGN KEY (user_id) REFERENCES users(id);
134
135 • ALTER TABLE transactions
136     ADD CONSTRAINT fk_transactions_users
137     FOREIGN KEY (user_id) REFERENCES users(id),
138     ADD CONSTRAINT fk_transactions_cards
139     FOREIGN KEY (card_id) REFERENCES credit_cards(id),
140     ADD CONSTRAINT fk_transactions_companies
141     FOREIGN KEY (business_id) REFERENCES companies(company_id);
142
143 ----- Ejercicio
```

100% 62:141

Action Output

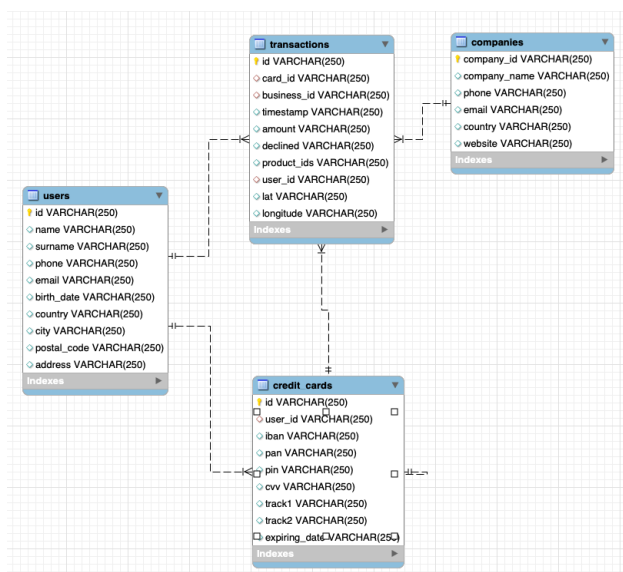
	Time	Action	Response
7	12:26:34	ALTER TABLE credit_cards ADD CONSTRAINT fk_credit_cards_...	5000 row(s)...
8	12:26:45	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_...	100000 row(s)...

Operaciones realizadas:

- Importé los datos de los archivos .CSV mediante sentencias LOAD DATA LOCAL INFILE.
- En el caso de users, cargué dos archivos CSV en una misma tabla comprobando q quedaba la suma de los 2
- Verifiqué la correcta carga de cada tabla mediante consultas SELECT COUNT(*).
- Creé entidad referencial entre tablas.

Resultado clave:

- Todas las tablas contienen los datos importados desde los archivos CSV correspondientes.



Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
145 • SELECT *
146 FROM users
147 WHERE id IN (
148     SELECT user_id
149     FROM transactions
150     GROUP BY user_id
151     HAVING COUNT(*) > 80
152 );
153
```

100% 6:152

Result Grid

id	name	surname	phone	email	birth_date	country
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kir
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland
NULL	NULL	NULL	NULL	NULL	NULL	NULL

users 9

Action Output

	Time	Action	Response
✓ 9	12:32:29	SELECT * FROM users WHERE id IN (SELECT user_id FROM...	4 row(s) retur...

Operaciones realizadas:

- Usé una subconsulta sobre la tabla transactions.
- Agrupé las transacciones por user_id.
- Filtré los usuarios que tienen más de 80 transacciones.
- Seleccioné los registros correspondientes en la tabla users.

Resultado clave:

- Se obtienen los usuarios que han realizado más de 80 transacciones.

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
156 • SELECT cc.iban,
157         ROUND(AVG(t.amount),2) AS media_amount
158 FROM transactions t
159 JOIN credit_cards cc
160     ON t.card_id = cc.id
161 JOIN companies co
162     ON t.business_id = co.company_id
163 WHERE co.company_name = 'Donec Ltd'
164 GROUP BY cc.iban;
165
```

100% 66:154

Result Grid Filter Rows: Search Export:

iban	media_amount
XX776752917845952975555640	257.37
XX413827362289719304908990	139.59
XX347787246070769610780308	240.41
XX688768436543090894854602	188.58
MK28368851538688349	439.39

Result 11

Action Output

	Time	Action	Response
✓ 11	12:35:08	SELECT cc.iban, ROUND(AVG(t.amount),2) AS media_amount...	371 row(s) re...

Operaciones realizadas:

- Se relaciona la tabla transactions con credit_cards mediante el identificador de tarjeta.
- Se relaciona la tabla transactions con companies mediante el identificador de empresa.
- Se filtran las transacciones correspondientes a la empresa Donec Ltd..
- Se agrupan los resultados por IBAN y se calcula la media del campo amount.

Resultado clave:

- Se obtiene la media del importe de las transacciones para cada IBAN asociado a la empresa Donec Ltd..



Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en *si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo*. Partiendo de esta tabla responde:

Ejercicio 1

¿Cuántas tarjetas están activas?

```
169 • CREATE TABLE credit_card_status AS
170 WITH ranked_transactions AS (
171     SELECT
172         id,
173         card_id,
174         declined,
175         timestamp,
176         ROW_NUMBER() OVER (
177             PARTITION BY card_id
178             ORDER BY timestamp DESC
179         ) AS row_num
180     FROM transactions
181 )
182 SELECT
183     id,
184     card_id,
185     declined,
186     timestamp,
187     row_num
188 FROM ranked_transactions
189 WHERE row_num IN (1, 2, 3);
190
191
192
```

00% 18:167

Time	Action	Response
15 12:47:08	SELECT * FROM sprint04.credit_card_status LIMIT 0, 50000	15000 row(s) returned

```
197 -- ¿Cuántas tarjetas están activas?
198 • SELECT COUNT(card_id) AS 'Núm. tarjetas activas'
199 FROM (
200     SELECT card_id
201     FROM credit_card_status
202     GROUP BY card_id
203     HAVING SUM(declined) < 3
204 ) AS credit_card_active;
205
206
```

100% 30:204

Result Grid

Núm. tarjetas activas
4995

Result 23

Time	Action	Response
40 13:15:54	SELECT COUNT(card_id) AS 'Núm. tarjetas activas' FROM (SELECT card_id FRO...	1 row(s) returned

El ejercicio solicita crear una nueva tabla que refleje el estado de las tarjetas de crédito en función de sus tres últimas transacciones.

La tabla representa una instantánea del estado de las tarjetas en un momento determinado, ya que cualquier transacción posterior no modifica los datos almacenados.

Para resolver el ejercicio, es necesario analizar el resultado (declined) de las tres transacciones más recientes asociadas a cada tarjeta y clasificar su estado como activa o inactiva según el criterio indicado.

En un entorno real, esta información podría obtenerse mediante una vista o se podría añadir a la tabla un proceso automático de actualización con triggers para mantenerla siempre actualizada.

En este caso, implemento la solución solicitada en el enunciado.

Operaciones realizadas:

- Ordené las transacciones por tarjeta y fecha utilizando ROW_NUMBER().
- Identifiqué las tres últimas transacciones de cada tarjeta.
- Creé la tabla credit_card_status con dichas transacciones para determinar el estado de cada tarjeta.
- Teniendo en cuenta la nueva tabla de “status” conté la cantidad de tarjetas Activas.

Resultado Clave:

- La tabla credit_card_status contiene las tres últimas transacciones de cada tarjeta y permite identificar su estado según el valor de declined.
- Dar respuesta a la consulta.



Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:



Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

Operaciones realizadas:

- Cree tabla intermedia para poder vincular transacciones y productos

```
216 • CREATE TABLE transaction_products (  
217     transaction_id VARCHAR(250) NOT NULL,  
218     product_id VARCHAR(250) NOT NULL,  
219     PRIMARY KEY (transaction_id, product_id)  
220 );  
221
```

100% 3:220

Action Output

	Time	Action
✓ 49	22:08:46	CREATE TABLE transaction_products (transaction_id VARCHAR(250) NOT NULL, prod

- Cree relación

```
222 • ALTER TABLE transaction_products  
223     ADD CONSTRAINT fk_tp_transactions  
224     FOREIGN KEY (transaction_id) REFERENCES transactions(id);  
225  
226 • ALTER TABLE transaction_products  
227     ADD CONSTRAINT fk_tp_products  
228     FOREIGN KEY (product_id) REFERENCES products(id);  
229
```

100% 3:229

Action Output

	Time	Action
✓ 50	22:10:01	ALTER TABLE transaction_products ADD CONSTRAINT fk_tp_transactions FOREIGN KEY (transaction_id) REFERENCES transactions(id)
✓ 51	22:10:06	ALTER TABLE transaction_products ADD CONSTRAINT fk_tp_products FOREIGN KEY (product_id) REFERENCES products(id)

- incluí los datos en la tabla intermedia valiéndome de tabla Jason para poder separar los productos que originalmente estaban incluidos en una lista

```

230 • INSERT INTO transaction_products (transaction_id, product_id)
231     SELECT
232         t.id,
233         jt.product_id
234     FROM transactions t
235     JOIN JSON_TABLE(
236         CONCAT('["', REPLACE(t.product_ids, ',', '","'), "']'),
237         '$[*]' COLUMNS (product_id VARCHAR(250) PATH '$')
238     ) jt
239     JOIN products p
240         ON p.id = jt.product_id
241     WHERE t.product_ids IS NOT NULL
242         AND t.product_ids <> '';
243

```

100% 1:229

Action Output

	Time	Action
✓ 54	22:14:48	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.product_id FROM transactions

- Resolví la cantidad de productos vendidos utilizando la misma tabla “intermedia” que contiene el nro de producto y el de transaccional realizada.

```

244 • SELECT
245     product_id AS producto,
246     COUNT(*) AS veces_vendido
247 FROM transaction_products
248 GROUP BY product_id;
249

```

100% 21:248

Result Grid

producto	veces_vendido
10	992
100	987
11	1048
12	994
13	1070
14	957
15	1026
16	1051

Result 28

Action Output

	Time	Action	Response
✓ 85	12:43:23	SELECT product_id AS producto, COUNT(*) AS ve...	100 row(s) returned